

Documentație proiect ISOC

1. Nume proiect și membrii echipei

Nume proiect: Pet Care Reminder App

Descriere scurtă: aplicație web bazată pe microservicii pentru gestionarea activităților de îngrijire a animalelor de companie.

Utilizatorii pot crea conturi, pot adăuga animale, pot crea remindere pentru hrană, baie, plimbare, vaccin, medicamente sau vizite la medicul veterinar și pot vizualiza notificările generate pentru activitățile programate.

Membrii echipei:

- Rancov Larisa
- Șerban Alexia
- Raț Ioan-Paul

2. Scurtă descriere și diagrama UC

Actorul principal al aplicației este utilizatorul. Acesta poate crea un cont simplu, poate adăuga animale de companie, poate gestiona reminderele asociate acestora și poate vizualiza notificările generate pentru activitățile programate. Al patrulea microserviciu, Notification Service, separă partea de notificări de logica reminderelor, ceea ce face arhitectura mai clară și mai apropiată de o aplicație reală.

Cazuri de utilizare principale:

- Creare utilizator
- Adăugare animal de companie
- Vizualizare animale
- Creare reminder
- Vizualizare remindere active
- Marcare reminder ca realizat
- Generare notificare pentru reminder
- Vizualizare notificări primite

Diagrama cazurilor de utilizare (Use Case)

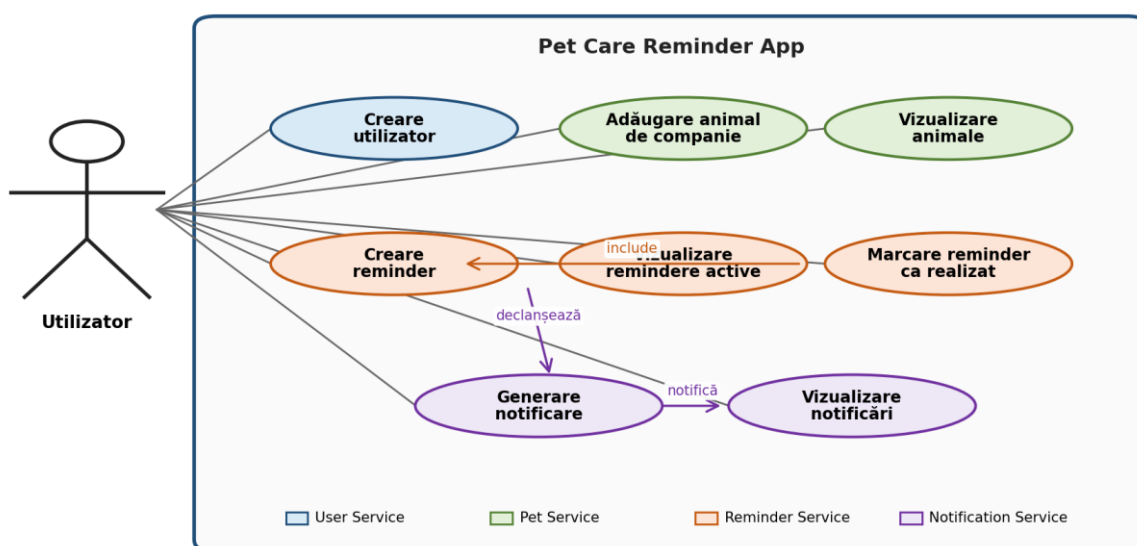


Figura 1. Diagrama cazurilor de utilizare

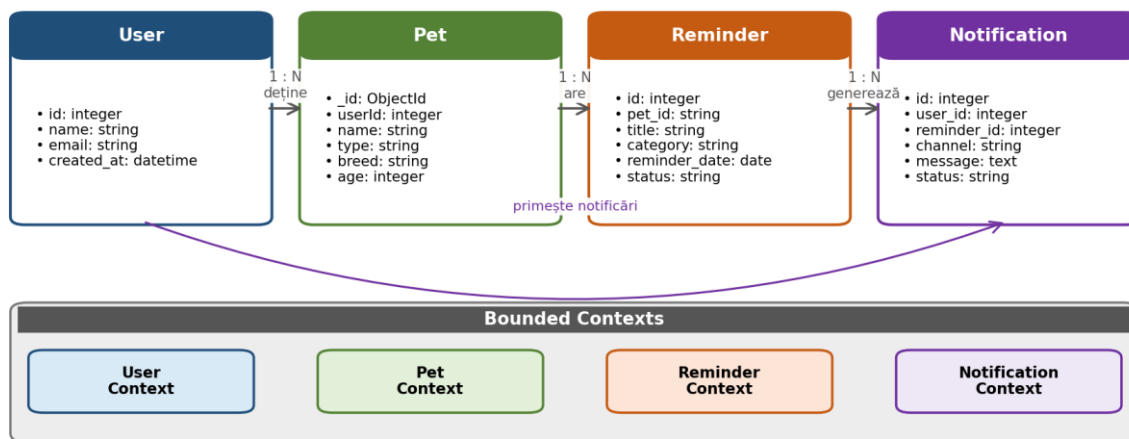
3. Modelul domeniului și contextele delimitate

Domeniul aplicației este reprezentat de gestionarea activităților de îngrijire pentru animalele de companie. Entitățile principale sunt User, Pet, Reminder și Notification. Relațiile dintre ele sunt: un utilizator poate avea mai multe animale, un animal poate avea mai multe remindere, iar un reminder poate genera una sau mai multe notificări. Notificările aparțin utilizatorului și pot fi urmărite separat față de reminderul inițial.

Bounded contexts identificate:

- User Context - gestionează utilizatorii și datele de contact
- Pet Context - gestionează animalele de companie
- Reminder Context - gestionează reminderele asociate animalelor
- Notification Context - gestionează notificările generate pe baza reminderelor

Modelul domeniului și contextele delimitate



Fiecare context este implementat ca microserviciu separat și are propria bază de date.

Figura 2. Modelul domeniului cu patru contexte delimitate

4. Descriere microservicii

Aplicația conține patru microservicii. Fiecare microserviciu are propria responsabilitate, propriul API REST și propria bază de date. Astfel se respectă principiul separării responsabilităților și cerința ca microserviciile să folosească baze de date diferite.

4.1 User Service

a. Funcționalitate

User Service este responsabil pentru gestionarea utilizatorilor aplicației. Acesta oferă funcționalități de creare utilizator, listare utilizatori și obținere utilizator după ID. Serviciul reprezintă sursa principală pentru datele de identificare și contact ale utilizatorului.

b. Descriere API, protocol de acces, punct de acces

- Protocol de acces: HTTP/HTTPS
- Punct de acces local: `http://localhost:3001`
- Punct de acces cloud AWS prin Nginx: `http://<EC2-PUBLIC-IP>/api/users` sau `https://<domeniu>/api/users`
- Endpoint-uri: `POST /users`, `GET /users`, `GET /users/{id}`

Exemplu request pentru `POST /users`: `{ "name": "Larisa", "email": "larisa@email.com" }`

c. Structura bazei de date

User Service folosește PostgreSQL, o bază de date relațională gratuită și open-source. Tabelul principal este `users` și conține câmpurile `id`, `name`, `email` și `created_at`. În cloud, PostgreSQL rulează într-un container Docker separat, cu volum persistent.

d. Tehnologii folosite

- Node.js
- Express.js
- PostgreSQL
- pg library
- Docker
- REST API

4.2 Pet Service

a. Funcționalitate

Pet Service gestionează animalele de companie ale utilizatorilor. Acesta permite adăugarea, listarea și ștergerea animalelor, precum și filtrarea lor după userId. Reminder Service poate apela Pet Service pentru a verifica existența animalului înainte de crearea unui reminder.

b. Descriere API, protocol de acces, punct de acces

- Protocol de acces: HTTP/HTTPS
- Punct de acces local: `http://localhost:3002`
- Punct de acces cloud AWS prin Nginx: `http://<EC2-PUBLIC-IP>/api/pets` sau `https://<domeniu>/api/pets`
- Endpoint-uri: POST /pets, GET /pets, GET /pets/{id}, GET /pets/user/{userId}, DELETE /pets/{id}

Exemplu request pentru POST /pets: { "userId": 1, "name": "Max", "type": "dog", "breed": "Golden Retriever", "age": 3, "notes": "Are nevoie de plimbare zilnică." }

c. Structura bazei de date

Pet Service folosește MongoDB Community Edition, o bază de date NoSQL gratuită și cunoscută. Colecția principală este pets și conține câmpuri precum _id, userId, name, type, breed, age și notes. În cloud, MongoDB rulează într-un container Docker separat, cu volum persistent.

d. Tehnologii folosite

- Node.js
- Express.js
- MongoDB Community Edition
- Mongoose
- Docker
- REST API

4.3 Reminder Service

a. Funcționalitate

Reminder Service gestionează reminder-ele asociate animalelor de companie. Acesta permite crearea reminderelor, afișarea reminderelor active, filtrarea reminderelor după animal și marcarea lor ca realizate. După crearea unui reminder sau atunci când un reminder devine scadent, serviciul poate trimite o cerere către Notification Service pentru generarea unei notificări.

b. Descriere API, protocol de acces, punct de acces

- Protocol de acces: HTTP/HTTPS
- Punct de acces local: `http://localhost:3003`
- Punct de acces cloud AWS prin Nginx: `http://<EC2-PUBLIC-IP>/api/reminders` sau `https://<domeniu>/api/reminders`
- Endpoint-uri: POST /reminders, GET /reminders, GET /reminders/active, GET /reminders/pet/{petId}, PUT /reminders/{id}/done, DELETE /reminders/{id}

Exemplu request pentru POST /reminders: { "petId": "665f1a2b1c45a90012dabc11", "title": "Vaccin anual", "description": "Vaccin pentru Max", "category": "vaccine", "reminderDate": "2026-06-20", "status": "active" }

c. Structura bazei de date

Reminder Service folosește MySQL Community Server, o bază de date relațională gratuită și foarte utilizată. Tabelul principal este reminders și conține câmpurile id, pet_id, title, description, category, reminder_date, status și created_at. În cloud, MySQL rulează într-un container Docker separat, cu volum persistent.

d. Tehnologii folosite

- Node.js
- Express.js
- MySQL Community Server
- mysql2 library
- Docker
- REST API

4.4 Notification Service

a. Funcționalitate

Notification Service este microserviciul adăugat pentru extinderea proiectului. Acesta gestionează notificările generate din remindere. Serviciul primește cereri de la Reminder Service, creează notificări cu status pending, permite listarea notificărilor pentru un utilizator și permite marcarea unei notificări ca sent sau read. Pentru demo, trimiterea notificării este simulată prin salvarea notificării și schimbarea statusului.

b. Descriere API, protocol de acces, punct de acces

- Protocol de acces: HTTP/HTTPS
 - Punct de acces local: `http://localhost:3004`
 - Punct de acces cloud AWS prin Nginx: `http://<EC2-PUBLIC-IP>/api/notifications` sau `https://<domeniu>/api/notifications`
 - Endpoint-uri: POST /notifications, GET /notifications, GET /notifications/user/{userId}, GET /notifications/reminder/{reminderId}, PUT /notifications/{id}/sent, PUT /notifications/{id}/read, DELETE /notifications/{id}
- Exemplu request pentru POST /notifications: { "userId": 1, "reminderId": 10, "channel": "in-app", "message": "Reminder: vaccin anual pentru Max", "status": "pending" }

c. Structura bazei de date

Notification Service folosește SQLite, o bază de date gratuită, open-source și foarte ușor de configurat. Baza de date este un fișier persistent, montat într-un volum Docker. Tabelul principal este notifications și conține câmpurile id, user_id, reminder_id, channel, message, status, created_at, sent_at și read_at.

d. Tehnologii folosite

- Node.js
- Express.js
- SQLite
- sqlite3 library
- Docker
- REST API

4.5 Structura comună a codului pentru microservicii

Pentru a evidenția arhitectura modulelor de cod, fiecare microserviciu este organizat după aceeași structură internă. Logica aplicației este separată clar de definirea rutelor HTTP și de accesul la baza de date.

- routes/ - definește endpoint-urile REST expuse de microserviciu
- controllers/ - primește request-ul, validează datele și apelează logica de business
- services/ - conține logica principală a microserviciului
- models/ sau repositories/ - definește structurile de date și operațiile de acces la baza de date
- db/ sau config/ - conține configurarea conexiunii la baza de date și variabilele de mediu
- Dockerfile - descrie modul de construire a containerului pentru deployment

Microserviciile nu accesează direct bazele de date ale altor microservicii. Comunicarea între ele se face prin API-uri REST. De exemplu, Reminder Service poate apela Pet Service pentru verificarea existenței animalului, iar apoi poate apela Notification Service pentru generarea notificării.

5. Modelul de instalare

Aplicația va fi instalată la un furnizor de cloud folosind containere Docker. Furnizorul ales este AWS, iar modelul de instalare este: o instanță EC2 Ubuntu, Docker Compose și Nginx reverse proxy.

a. Modelul de instalare folosit

- AWS EC2 Free Tier / instanță Ubuntu pentru rularea aplicației în cloud
- Docker și Docker Compose pentru pornirea microserviciilor și a bazelor de date
- Nginx ca reverse proxy și server pentru frontend-ul web
- Containere Docker pentru cele patru microservicii: User Service, Pet Service, Reminder Service și Notification Service
- Baze de date separate, gratuite și open-source: PostgreSQL, MongoDB Community Edition, MySQL Community Server și SQLite
- Volume Docker pentru păstrarea datelor după repornirea containerelor
- GitHub pentru codul sursă și deployment manual simplu pe EC2
- Docker logs pentru verificarea erorilor în timpul prezentării

b. Componentele infrastructurii de instalare și tipul de resursă/serviciu

- AWS EC2 - mașină virtuală cloud pe care rulează toate containerele aplicației
- AWS Security Group - reguli de acces pentru porturile necesare, de exemplu 80/443 și SSH
- Nginx - reverse proxy pentru rutele /api/users, /api/pets, /api/reminders și /api/notifications; poate servi și frontend-ul static
- Frontend Web - interfața utilizatorului, realizată cu HTML/CSS/JavaScript și servită prin Nginx
- User Service - container Node.js + Express
- Pet Service - container Node.js + Express
- Reminder Service - container Node.js + Express
- Notification Service - container Node.js + Express
- PostgreSQL - baza de date pentru User Service
- MongoDB Community Edition - baza de date pentru Pet Service
- MySQL Community Server - baza de date pentru Reminder Service
- SQLite - baza de date pentru Notification Service
- Docker Compose - fișierul care definește serviciile, rețelele interne și volumele persistente

Model de instalare (Deployment) - AWS EC2 + Docker Compose

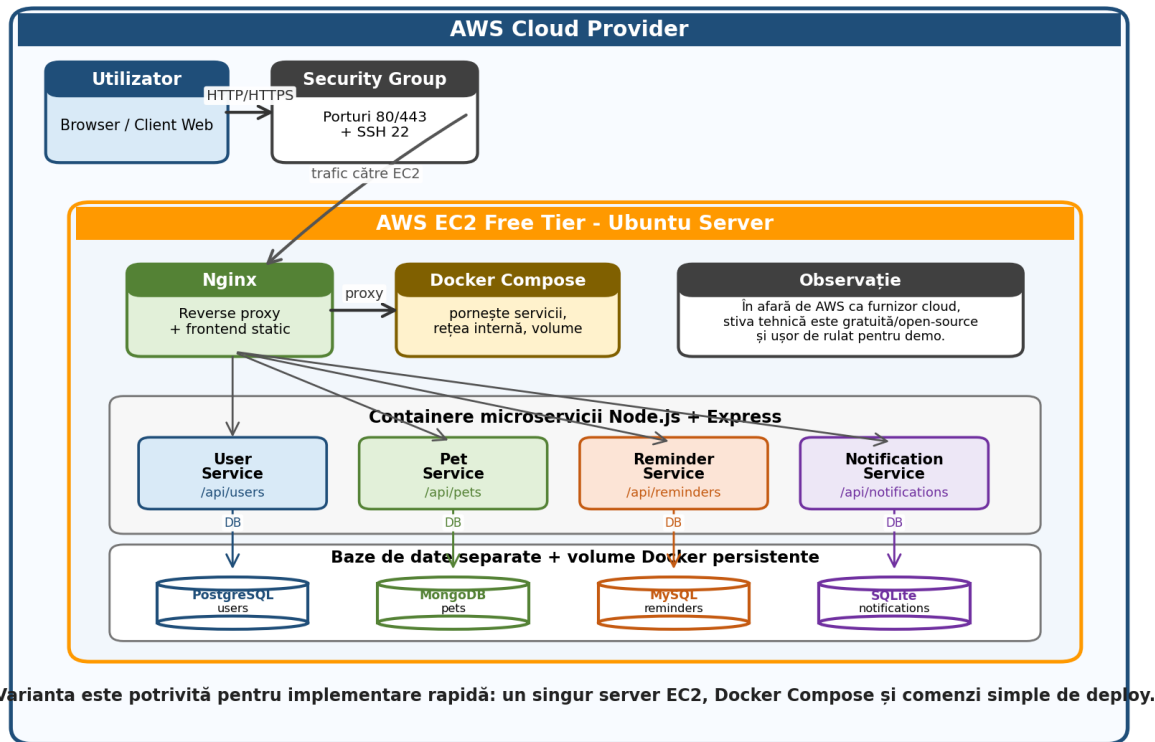


Figura 3. Modelul de instalare AWS cu EC2, Docker Compose, Nginx, microservicii și baze de date separate

6. Reprezentarea arhitecturii de ansamblu

Arhitectura generală poziționează frontend-ul în fața celor patru microservicii. Fiecare microserviciu are propria structură internă de cod și propria bază de date. User Service gestionează utilizatorii, Pet Service gestionează animalele, Reminder Service gestionează programările, iar Notification Service gestionează notificările generate din remindere.

Flux principal de utilizare:

- Utilizatorul creează un cont simplu în User Service.
- Utilizatorul adaugă un animal de companie în Pet Service.
- Utilizatorul creează un reminder pentru animal în Reminder Service.
- Reminder Service verifică, dacă este necesar, existența animalului prin Pet Service.
- Reminder Service trimite o cerere către Notification Service pentru generarea notificării.
- Notification Service salvează notificarea în SQLite și îi setează statusul pending, sent sau read.
- Utilizatorul vizualizează reminderele active și notificările primite, apoi poate marca un reminder ca done.

Arhitectura de ansamblu: module de cod, API-uri și date

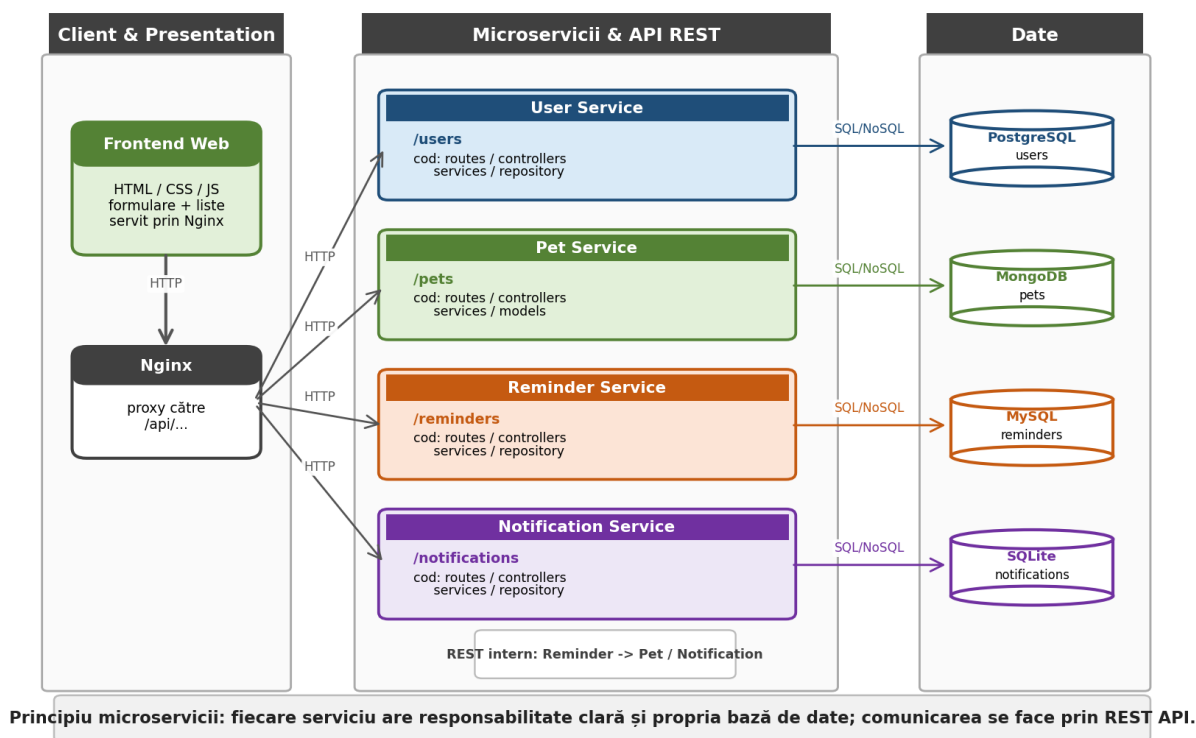


Figura 4. Arhitectura de ansamblu cu poziționarea modulelor de cod și a datelor

7. Contribuția fiecărui membru al echipei la realizarea proiectului

- Rancov Larisa - implementare User Service, configurare PostgreSQL, implementare Notification Service, configurare SQLite și documentare API pentru User Service și Notification Service.
- Șerban Alexia - implementare Pet Service, configurare MongoDB, documentare API Pet Service și integrarea cu Reminder Service pentru validarea animalelor.
- Raț Ioan-Paul - implementare Reminder Service, configurare MySQL, realizare frontend, integrare între servicii și deployment AWS pe EC2.
- Toți membrii echipei - definirea ideii, alegerea arhitecturii pe microservicii, realizarea documentației, testarea fluxului principal și pregătirea prezentării finale.